REPORT 003: EPOCH 3: MULTI-DEVICE MIDI REMAPPING ENGINE, RESPONSE CURVES & SOFT TAKEOVER STATE MACHINE

2026-11-04 00:20:00 | Epoch 3 of 20: src/mapping_engine.rs (CC<->Note transformations, response curves, toggle/momentary buttons, and Catch/Jump/Pickup Soft Takeover)

Operator: Gemini 4 Argon (Medium) · Follows: report-002

Prompt

```
Epoch 3 of 20: Implement the refactored MIDI Remapping Engine in
↪ src/mapping_engine.rs supporting simultaneous per-device mapping pipelines:
↪ CC-to-CC, Note-to-CC, CC-to-Note, and Note-to-Note transformations, per-control
↪ output MIDI channel (1-16), min/max output range scaling, polarity inversion,
↪ response curves (Linear, Exponential, Logarithmic, S-Curve), Toggle vs. Momentary
↪ button state machines with customizable on/off values, and all three Soft
↪ Takeover modes (Catch, Jump, Pickup) with real-time physical-vs-target delta
↪ calculation so physical knob/slider jumps never cause sudden parameter spikes.
↪ Add comprehensive Rust unit tests (with CARGO_BUILD_JOBS=2), git commit all
↪ changes, and compile reports/report-003.html from reports/report-003.md via
↪ /home/renick/claude/html-report-generator/build-report.sh --pdf and commit.
```

TL;DR

Implemented `src/mapping_engine.rs` providing simultaneous per-device MIDI remapping pipelines (`MultiDeviceMappingEngine` and `DeviceMappingPipeline`). Supports all four cross-message transformations (`CcToCc`, `NoteToCc`, `CcToNote`, `NoteToNote`), per-control output MIDI channels (`1..=16`), `min_value..=max_value` range scaling, polarity inversion (`invert`), four continuous response curves (`Linear`, `Exponential`, `Logarithmic`, `SCurve`), `Toggle` vs. `Momentary` button state machines with customizable `on_value`/`off_value`, a built-in `nanoKONTROL2 -> Akai MIDImix` compatibility preset, and all three Soft Takeover modes (`Catch`, `Jump`, `Pickup`) with real-time physical-vs-target delta telemetry (`SoftTakeoverStatus`). Verified by 30 passing automated tests (`24` Rust unit/integration tests + `6` Node.js SDK tests).

1. Plain-Language Overview & Epoch 3 Goals

Physical MIDI controllers have fixed factory hardware behaviors: a Korg `nanoKONTROL2` sends Control Change (**CC**, a standard MIDI message type used for continuous sliders and switches) numbers `0..71` on MIDI Channel 1, whereas an Akai `MIDI Mix` sends **Note On/Off** messages (traditionally used for musical notes) when you press its buttons. Furthermore, physical knobs and sliders are **non-motorized**: if you turn a physical knob to `20`, and then load a software preset where that knob's saved value is `80`, touching the knob would normally cause the sound to jump abruptly from `80` down to `21`—creating a jarring audio glitch during a live performance.

In **Epoch 3 of 20**, we implemented `src/mapping_engine.rs`, upgrading the legacy single-pipeline TypeScript engine into a simultaneous multi-device Rust remapping engine:

1. **Simultaneous Per-Device Mapping Pipelines (`MultiDeviceMappingEngine` & `DeviceMappingPipeline`)**: Each connected controller runs its own isolated remapping pipeline keyed by its hardware fingerprint, using a composite lookup key (`CompositeMidiKey`, e.g. `cc:16` vs `note:16`) so CC and Note messages sharing the same `0..=127` index never collide.

2. **All 4 Cross-Message Transformations (`MessageTransformation`)**: Any physical control can translate between **CC-to-CC**, **Note-to-CC**, **CC-to-Note**, and **Note-to-Note** on any per-control output MIDI channel (`1..=16`). This also powers a built-in `nanoKONTROL2 -> Akai MIDImix` compatibility preset (`DeviceMappingConfig::nanokontrol2_to_midimix_emulation`).
3. **Response Curves, Range Scaling & Polarity Inversion**: Continuous controls scale their output to any `[min_value, max_value]` range (`0..=127`), optionally invert polarity (`0..127 -> 127..0`), and shape their response feel across 4 transfer curves (`Linear`, `Exponential`, `Logarithmic`, and `SCurve`).
4. **Toggle vs. Momentary Button State Machines**: Buttons can either latch ON/OFF on each press (`Toggle`) or stay active only while held down (`Momentary`), emitting customizable `on_value` and `off_value` payloads.
5. **All 3 Soft Takeover Modes (`Catch`, `Jump`, `Pickup`) with Real-Time Delta Telemetry**: Prevents sudden parameter spikes after preset changes and computes the exact signed distance (`delta`), absolute distance (`abs_delta`), and required movement direction (`MoveUp`, `MoveDown`, `Matched`) between the physical knob and the software target on every hardware event.

Plain-English Concepts Mapped to Code Components

Plain-English Concept	What It Does for the Musician	Exact Code Component
Cross-Message Translation	Translates `CC<->CC`, `Note<->CC`, `CC<->Note`, and `Note<->Note` on any output channel (`1..=16`)	`src/mapping_engine.rs` (`MessageTransformation`, `ControlMappingEntry`, `CompositeMidiKey`)
Feel & Range Shaping	Shapes knob/fader travel with `Linear`, `Exponential`, `Logarithmic`, or `SCurve` curves, `[min, max]` scaling, and `invert`	`src/mapping_engine.rs` (`ResponseCurve`, `ResponseCurve::transform_value`)
Button Latching Logic	Handles `Toggle` (latch on press) vs `Momentary` (active while held) with custom `on_value`/`off_value`	`src/mapping_engine.rs` (`ButtonBehavior`, `ButtonStateChangeEvent`)
Soft Takeover Anti-Spike Protection	Locks controls after preset changes until the physical position catches or crosses the software target	`src/mapping_engine.rs` (`SoftTakeoverMode`, `SoftTakeoverTracker`, `SoftTakeoverStatus`)
Simultaneous Multi-Device Engine	Runs independent mapping pipelines per hardware controller without cross-talk	`src/mapping_engine.rs` (`DeviceMappingPipeline`, `MultiDeviceMappingEngine`)

2. Phase 1: Cross-Message Transformations, Response Curves & Button State Machines (‘src/mapping_engine.rs‘)

Per-Control Transformation Pipeline

`DeviceMappingPipeline` processes every incoming hardware event (`evaluate_message` and `evaluate_raw_packet`) through a deterministic pipeline:

- **Composite Key Disambiguation (`CompositeMidiKey`)**: Pairs `(MidiMessageType, u8)` (`"cc:16"` vs `"note:16"`) so hybrid controllers like the Akai `MIDI Mix` (which has both `CC 16` on Track 1 Knob 1 and `Note 16` on Track 6 Mute) never suffer lookup collisions.
- **All 4 Cross-Message Transformations (`MessageTransformation`)**: Each `ControlMappingEntry` defines `input_message_type` and `output_message_type`, supporting `CcToCc`, `NoteToCc`, `CcToNote`, and `NoteToNote` on any `output_channel` (`1..=16`), plus

optional `input_channel` filtering.

- **4 Continuous Response Curves (`ResponseCurve`)**: Evaluates normalized input $t \in [0, 1]$ (after optional polarity inversion $t \leftarrow 1 - t$) through `Linear` ($f(t) = t$), `Exponential` ($f(t) = t^2$), `Logarithmic` ($f(t) = \ln(1 + 9t) / \ln(10)$), or `SCurve` (Hermite smoothstep $f(t) = 3t^2 - 2t^3$), and scales the result into `[min_value, max_value]`.
- **Toggle vs. Momentary Button State Machines**: `Toggle` buttons flip their latched boolean state on press (`raw_value >= 64`) and ignore button release (`MappingEvaluation::ButtonReleaseIgnored`), whereas `Momentary` buttons emit `on_value` on press and `off_value` on release.
- **Built-in `nanoKONTROL2 -> MIDImix` Compatibility Preset (`DeviceMappingConfig::nanokontrol2_to_midi`)**: Translates Korg `nanoKONTROL2` Track Mute/Solo/Rec CCs (`CC 48..55`, `32..39`, `64..71`) and transport buttons (`CC 58`, `59`, `46`) into the exact Akai `MIDI Mix` Note numbers (`Notes 1..27`) and density CCs (`CC 17..61`).

3. Phase 2: Soft Takeover ('Catch', 'Jump', 'Pickup') & Real-Time Delta Calculation

Preventing Sudden Parameter Spikes on Non-Motorized Knobs and Sliders

When a preset is loaded via `set_target_value` or `load_preset_targets`, `SoftTakeoverTracker` compares the new software `target_value` against the hardware control's `last_physical_value`:

- `SoftTakeoverMode::Catch` **(Default)**: Locks the control (`engaged = false`) until the physical reading enters the threshold window (`|physical_value - target_value| <= threshold`, default `threshold = 3`). If a fast sweep jumps over the window without landing inside `[target - threshold, target + threshold]`, `Catch` remains locked.
- `SoftTakeoverMode::Pickup`: Unlocks when the physical reading enters the threshold window OR when consecutive physical samples cross over the target value (`(prev < target && curr >= target) || (prev > target && curr <= target)`). This detects the zero-crossing of `delta = physical - target` so rapid live fader sweeps never miss the pickup point.
- `SoftTakeoverMode::Jump`: Never locks (`engaged = true`); every hardware movement immediately updates the output to the physical value.
- **Real-Time Physical-vs-Target Telemetry (`SoftTakeoverStatus`)**: On every physical movement (even while locked via `MappingEvaluation::SoftTakeoverLocked`), the tracker computes `physical_value`, `target_value`, signed `delta` (`-127..=127`), `abs_delta` (`0..=127`), and `TakeoverDirection` (`MoveUp`, `MoveDown`, or `Matched`) for real-time GUI visual feedback.

Soft Takeover Modes Comparison

Soft Takeover Mode	Unlocks Inside Window ('\	phys - target\	<= T')?	Unlocks on Fast Zero-Crossing ('prev < target <= curr')?	Suppresses Parameter Jumps?	Primary Use Case
`Catch`	Yes	No (must land within threshold window)	Yes	Precision studio parameter tweaking		

Soft Takeover Mode	Unlocks Inside Window (‘\	phys - target\	<= T‘)?	Unlocks on Fast Zero-Crossing (‘prev < target <= curr‘)?	Suppresses Parameter Jumps?	Primary Use Case
`Pickup`	Yes	Yes (detects sign change of `delta`)	Yes	Fast live performance fader & knob sweeps		
`Jump`	Always unlocked	Always unlocked	No (immediate jump to physical value)	Instant macro / scratch overrides		

Tests

- `CARGO_BUILD_JOBS=2 cargo test` — 24 Rust unit and integration tests passing (`7` new tests in `mapping_engine::tests` covering all 4 message transformations, all 4 response curves, polarity inversion, min/max scaling, toggle/momentary custom on/off values, `Catch`/`Jump`/`Pickup` soft takeover with real-time delta, simultaneous multi-device pipelines, and the 51-control `nanoKONTROL2 -> MIDImix` emulation preset), 0 failures
- `npm test` (`node --test js/*.test.js`) — 6 Node.js SDK unit tests passing, 0 failures

QA

```
`30 tests passing (24 Rust + 6 Node.js)`

`All 4 cross-message transformations (CC->CC, Note->CC, CC->Note, Note->Note) verified`

`All 4 response curves (Linear, Exponential, Logarithmic, S-Curve) & polarity inversion verified`

`All 3 Soft Takeover modes (Catch, Jump, Pickup) & real-time delta telemetry verified`

`0 warnings, 0 failures`
```

All Epoch 3 deliverables compiled cleanly with `CARGO_BUILD_JOBS=2` and passed all Rust and Node.js test suites.

Files touched

- ‘src/mapping_engine.rs‘ — Implemented ‘MessageTransformation‘, ‘CompositeMidiKey‘, ‘ResponseCurve‘, ‘SoftTakeoverMode‘, ‘TakeoverDirection‘, ‘SoftTakeoverStatus‘, ‘SoftTakeoverTracker‘, ‘ControlMappingEntry‘, ‘DeviceMappingConfig‘, ‘ProcessedMidiMessage‘, ‘MappingEvaluation‘, ‘DeviceMappingPipeline‘, and ‘MultiDeviceMappingEngine‘ + 7 unit test suites
- ‘src/midi_io.rs‘ — Added ‘RawMidiPacket::note_off‘ and ‘RawMidiPacket::decode‘ (normalizing ‘0x80‘ Note Off to velocity ‘0‘)
- ‘src/devices.rs‘ — Derived ‘PartialOrd‘ and ‘Ord‘ on ‘MidiMessageType‘ for deterministic ‘BTreeMap‘ indexing

- ‘src/lib.rs‘ — Exported ‘pub mod mapping_engine‘
- ‘src/main.rs‘ — Integrated ‘MultiDeviceMappingEngine‘ into ‘NkEditorApp‘
- ‘js/nkeditor-sdk.js‘ — Added ‘RESPONSE_CURVES‘, ‘SOFT_TAKEOVER_MODES‘, and ‘transformContinuousValue‘
- ‘js/nkeditor-sdk.test.js‘ — Added unit tests verifying JavaScript response curves and range scaling match Rust ‘mapping_engine‘

Test coverage (30 total)

QA verdict

PASS

Report 003 | Generated 2026-11-04 00:20:00 | reports/report-003.html